ORIGINAL ARTICLE

Keyframe-Anchored Object Maps: Registering Late Zero-Shot 6D Pose Estimates in Asynchronous SLAM

Abstract

A zero-shot 6D pose estimator recognizes objects from CAD models alone but needs about a second per image. Running it on a second computer keeps SLAM real-time, yet its results arrive 0.7–6.6 s after capture, from another camera and clock, after loop closure may have corrected the map. This study presents a two-stage method that maps such late results consistently. In the first stage, asynchronous perception and capture-time registration, every result is registered with the SLAM pose interpolated at its capture time and expressed relative to the reference keyframe of that time. In the second stage, a keyframe-anchored object state map stores each object as an identifier, anchor keyframe, relative pose, existence probability, and class; a loop closure updates only keyframe poses, so every object follows the correction. On 61 replayed live runs, the median deviation of repeated observations drops from 9.67 cm (arrival-time pose) to 1.51 cm, and when the map changes in flight the share above 10 cm drops from 30.8% to 1.5%. A verification stage lets the recognizer abstain, cutting wrong answers per image sixfold at 0.24 s per image.

KEYWORDS

asynchronous sensor fusion, loop closure, object-level mapping, simultaneous localization and mapping, zero-shot 6D object pose estimation

1 | INTRODUCTION

Service and logistics robots, digital twins, and XR applications need *object-level* maps: the objects in a space, each with a 3D position and orientation in the map frame, kept up to date while the platform moves. Visual and LiDAR SLAM systems such as ORB-SLAM3 [1] and HDL-Graph-SLAM [2] estimate the sensor pose in real time and remove accumulated drift by loop closure. Zero-shot 6D object pose estimators such as SAM-6D [3], FoundPose [4], and GigaPose [5] estimate the pose of an object unseen in training from its CAD model alone.

The two do not combine by multiplying two transforms. Zero-shot estimators are slow, and an estimator that shares a computer with SLAM takes its time budget: in our system, SLAM pose delivery latency rose from 0.04 s to 2.5 s when a heavy process ran on the same computer, and a slower SLAM tracks less reliably and corrects its map more often. Fusion++ [6], which runs recognition and SLAM together, operates at 4–8 Hz overall. Running recognition on a second computer keeps SLAM real-time, but then each result arrives seconds after its image was captured, from a different camera and clock, and SLAM may correct its past keyframes by loop closure while the result is in flight.

Interpolating the trajectory at the capture timestamp, as a timestamped transform buffer such as ROS tf [7] does, places a late result correctly at the time of registration, but it stores the object in the map frame; after a loop closure moves the trajectory, the object stays behind. Figure 1 (panel 6) shows a real case: after a loop closure that moved keyframes by up to 3.05 m, 68 of 119 results stored this way lie more than 10 cm from their final object position, versus 6 of 119 with the method proposed here.

This study organizes the solution in two stages (Fig. 1). The contributions are:

- **Asynchronous perception and capture-time registration (stage A).** The SLAM computer streams, with every pose, the reference keyframe used to track it; the recognition computer registers each late 6D result with the SLAM pose interpolated at its capture time and expresses it relative to the reference keyframe of that time, waiting when the pose has not arrived. Capture timestamps, clocks, and the camera–camera extrinsic are aligned jointly.
- **Keyframe-anchored spatial object mapping (stage B).** Each object is stored as $\mathcal{O}_i = \{ID_i, K_i, T_{K_\{i} \leftarrow O_i\}, p_i, C_i\}$. A loop closure updates only keyframe poses, re-sent as keyframe lists, so all objects follow the correction while $T_{K_\{i} \leftarrow O_i\}$ stays unchanged; existence probabilities keep objects through absences.
- **A zero-shot recognizer that can abstain.** SAM-6D is kept unchanged and followed by verification of its whole set of 300 pose hypotheses against the color image; cluster-first verification fits this check into the real-time budget. An object map keeps every wrong answer, whereas a missed object is seen again, so precision is favored.
- **An evaluation** on YCB-Video [8], three further BOP datasets [25], and indoor runs that separates the effect of capture time, keyframe-relative storage, and the choice of keyframe.

2 | RELATED WORK

Zero-shot 6D pose estimation. CNOS [10] and SAM-6D [3] segment the image with SAM [11] or FastSAM [12], describe each proposal with DINOv2 [13] features, and match it against templates rendered from the CAD model. FoundPose [4], GigaPose [5], and FoundationPose [14] follow related designs. These methods output camera-frame poses for single images and return a pose for every queried object, whether or not it is present; benchmarks [25] score recall only, so the cost of a wrong answer is not measured. Render-and-compare refinement raises accuracy at several seconds per image. Our recognizer keeps SAM-6D and adds a decision on whether to answer.

Object SLAM and object maps. SLAM++ [16], CubeSLAM [17], and QuadricSLAM [18] use objects as landmarks to improve localization; CosyPose [19] aggregates multi-view object poses; Fusion++ [6], POCD [20], and Khronos [21] maintain object maps with existence or change models. We do not feed objects back into SLAM; SLAM estimation is unchanged, and CAD-defined objects follow its corrections.

Asynchronous perception. MaskFusion [22] runs segmentation asynchronously on a separate GPU of one computer with one camera and one clock. Edge-assisted mobile AR [23] offloads detection and compensates the returned boxes by image tracking, without a map. Transform libraries [7] interpolate poses at a requested time but do not revisit stored results after the trajectory is optimized. To the best of our knowledge, no prior system binds late, off-device recognition results to the SLAM keyframe of their capture time.

3 | OVERVIEW

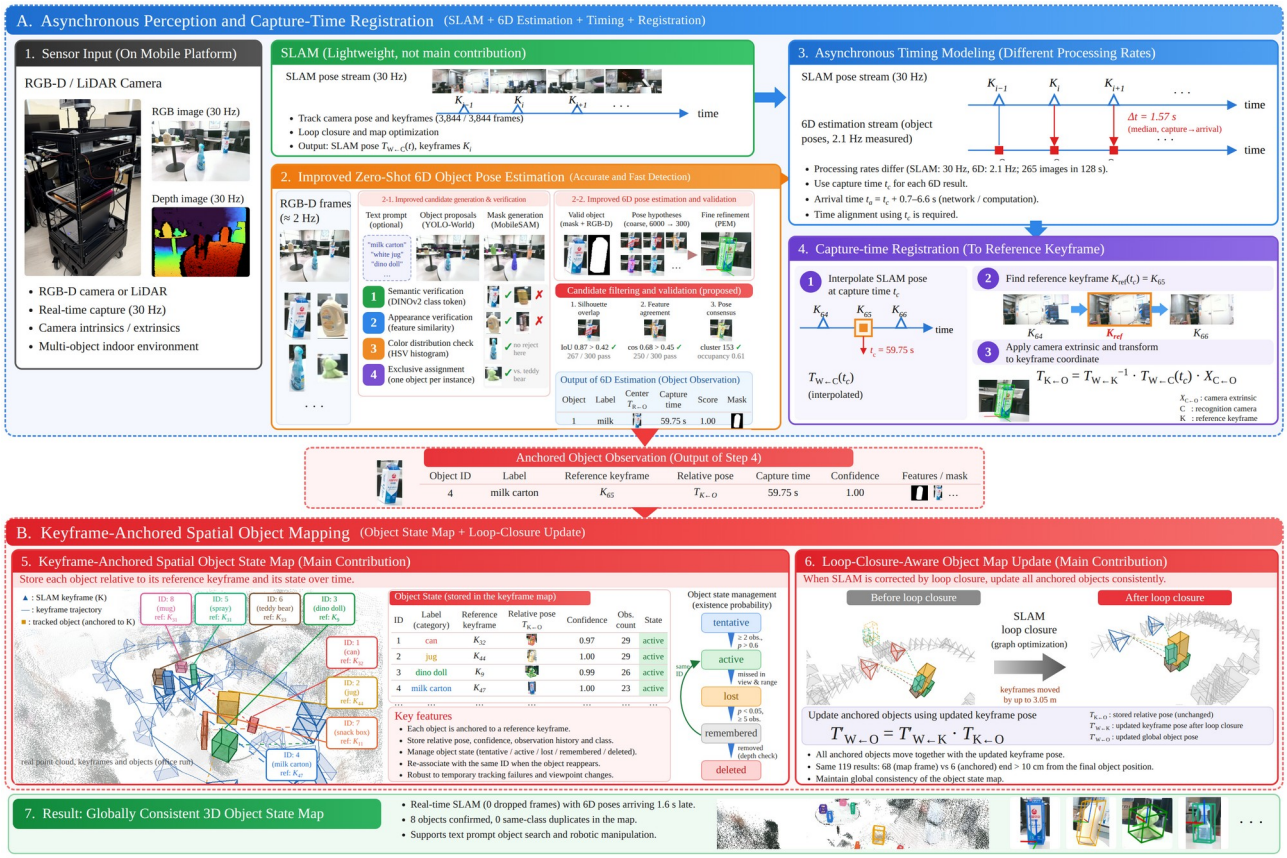


FIGURE 1 Overview of the method. Stage A (asynchronous perception and capture-time registration): 1, sensor input on the cart; SLAM stream of poses and keyframes; 2, zero-shot 6D estimation (the text-prompted variant used in the recorded runs); 3, asynchronous timing; 4, capture-time registration to the reference keyframe, producing an anchored object observation. Stage B (keyframe-anchored spatial object mapping): 5, object state map; 6, loop-closure-aware update. 7, result. All images, tables, and values are from recorded runs: panels 1–5 and 7 from the office figure-eight run, panel 6 from the large figure-eight run with a 3.05 m loop correction.

Figure 1 shows the method. Two computers are connected by a wired LAN. The SLAM computer runs ORB-SLAM3 (or HDL-Graph-SLAM) on its own RGB-D camera (or LiDAR); the recognition computer has a GPU and a second RGB-D camera. Stage A turns each 6D result into an *anchored object observation*: the object label, its capture time t_c, the reference keyframe $K_{ref}(t_c)$, the relative pose $T_{K \leftarrow O}$, a confidence, and its mask. Stage B accumulates these observations into a keyframe-anchored object state map and keeps the map consistent when SLAM corrects its keyframes. Notation: $T_{A \leftarrow B}$ maps coordinates from frame B to frame A; W is the map (world) frame; S and R are the SLAM and recognition cameras; $X = T_{S \leftarrow R}$ is the camera–camera extrinsic.

4 | ASYNCHRONOUS PERCEPTION AND CAPTURE-TIME REGISTRATION

4.1 | Sensor platform and time base



FIGURE 2 Data-collection cart with two hardware-synchronized RGB-D cameras (SLAM and recognition) and a 16-channel LiDAR.

The cart (Fig. 1, panel 1; Fig. 2) carries two RGB-D cameras (640 × 480, 30 Hz), hardware-synchronized by a sync cable, and a 16-channel LiDAR (10 Hz). The capture time of every frame is the timestamp the camera attaches to the frame metadata; recorder association stamps deviated from the true frame times by −2.1 s to +3.8 s because of frame drops, whereas metadata timestamps agree within 1.4 ms. The clock offset between the computers is measured by UDP round trips,

$$\Delta = t_r - (t_0 + (t_1 - t_0) / 2) \qquad (1)$$

with an error bound of half the round-trip time (3.08 ms measured). The remaining offset τ and the extrinsic X are estimated together from a calibration drive in three stages chosen by observability under planar motion: gravity from each trajectory plane; planar translation, yaw, and τ from relative motions over 0.5–4 s windows (AX = XB, Huber-weighted, τ scanned); and the height difference from the floor plane in each depth image. On the office figure-eight run the fit uses 7596 pairs with a median residual of 1.05 cm and τ = +2 ms.

4.2 | SLAM as a pose and keyframe stream

SLAM is an existing component; we change neither its estimation nor its map. Two additions make it usable by another computer. First, tracking stays within the camera period: with 2000 ORB features, result-preserving changes (removing copies for a disabled viewer, in-place aggregation, POPCNT descriptor distance, fewer locks, parallel pyramid extraction) reduce the mean tracking time from 24.7 ms to 15.4 ms and over-period frames from 170 to 0 of 7142, with the loop-return error unchanged (26.6 cm versus 26.5 cm, Fig. 3). Second, for every frame the SLAM computer sends a pose message (t, $T_{W \leftarrow S}$, K_{ref}, $T_{W \leftarrow K_\{ref\}}$), read in the tracking thread so that pose and reference keyframe refer to the same instant, and it sends the list of all live keyframes every 1 s, right after each map change, and again 0.5 s and 3 s later, because global bundle adjustment finishes seconds after the loop closure. A keyframe missing from a list is treated as culled.

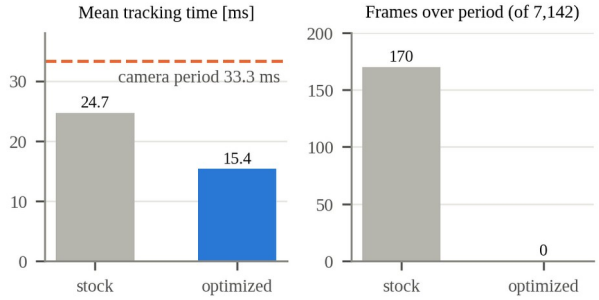


FIGURE 3 ORB-SLAM3 with 2000 features before and after the result-preserving optimizations.

4.3 | Zero-shot 6D estimation with hypothesis-set verification

A zero-shot estimator such as SAM-6D [3] returns a pose for every queried object, even one that is not in the image. In an object map a wrong answer persists, whereas a missed object is recovered when it is seen again; the recognizer (Fig. 1, panel 2) is therefore built to abstain. It keeps the SAM-6D front end and pose estimation model (PEM) without retraining: FastSAM-x [12] segments the whole image, each proposal is described by DINOv2 [13] features (ViT-S/14 instead of ViT-L/14), and SAM-6D's matching score selects proposals at 0.5. For each selected proposal, PEM forms 300 pose hypotheses and SAM-6D outputs the one with the best geometric score.

Instead of trusting that single hypothesis, we test the whole set against the color image, which the depth-based geometric score does not use. A hypothesis survives if (1) the silhouette of the CAD model projected with it overlaps the proposal mask with IoU ≥ 0.421 and (2) the texture rendered with it agrees with the image (score ≥ 0.450), which rejects shape-consistent but flipped poses. (3) The surviving hypotheses must agree: the largest cluster within 20° and 25 mm (symmetric objects folded about their axes) must hold at least a share o of them (o = 0.3 on YCB-Video, 0.5 on indoor runs). Its center is the answer and is checked again after refinement; if no hypothesis survives or they disagree, the recognizer does not answer and records the reason.

Rendering 300 hypotheses is the main cost. In *cluster-first* verification the hypotheses are grouped in pose space first (same 20° and 25 mm), one representative per cluster is rendered and tested, and consensus is weighted by cluster size; an object rejected for lack of consensus falls back to the full check. A median of 68 clusters replaces the 300 hypotheses, and verification time drops from 143 ms to 54 ms with 99% of answers unchanged (Section 7.3). The recorded runs of Sections 7.1–7.2 used an earlier, text-prompted variant (YOLO-World [15] boxes, MobileSAM [24] masks, and the same verification); registration uses only the capture time and the camera-frame pose and does not depend on the recognizer.

4.4 | Asynchronous timing

The two streams run at different rates (Fig. 1, panel 3). SLAM produces a pose for every frame at 30 Hz; the recognizer processed 265 images in 128 s (2.1 Hz) on the office run. Each 6D result keeps its capture time t_c and arrives at $t_a = t_c + \Delta t$, with Δt between 0.7 s and 6.6 s

across runs (median 1.57 s on the office run, 1.45 s over 61 live runs). Pose messages, keyframe lists, and results travel over the LAN and can arrive out of order; in particular, the SLAM pose for t_c may arrive after the result.

4.5 | Capture-time registration

Registration (Fig. 1, panel 4) has three steps. (1) The SLAM pose at t_c is interpolated between the two bracketing poses (SLERP for rotation, linear for translation); results whose bracketing poses are more than 0.25 s apart are discarded, and results whose poses have not yet arrived wait in a queue for up to 20 s. (2) The reference keyframe $K = K_{ref}(t_c)$ is the one sent with the bracketing pose nearest to t_c, together with its pose $T_{W\leftarrow K}(t_c)$ from the same message. (3) The extrinsic is applied and the object is expressed in that keyframe:

$$T_{K\leftarrow O} = T_{W\leftarrow K}(t_c)^{-1} \cdot T_{W\leftarrow S}(t_c) \cdot X \cdot T_{R\leftarrow O} \qquad (2)$$

Because $T_{W\leftarrow S}(t_c)$ and $T_{W\leftarrow K}(t_c)$ come from the same tracking state, (2) is unaffected by any correction applied to K after t_c, including corrections made while the result was in flight. Anchoring to the keyframe current at *arrival* would pair a pre-correction camera pose with a post-correction keyframe pose. The output is the anchored object observation of Fig. 1 (label, K, $T_{K\leftarrow O}$, t_c, confidence, mask).

5 | KEYFRAME-ANCHORED SPATIAL OBJECT MAPPING

5.1 | Keyframe-anchored object state map

Each object in the map (Fig. 1, panel 5) is a record

$$O_i = \{\, ID_i,\ K_i,\ T_{K_{\{i}\leftarrow O_i\}},\ p_i,\ C_i \,\} \qquad (3)$$

with identifier ID_i, anchor keyframe K_i, relative pose $T_{K_{\{i}\leftarrow O_i\}}$, existence probability p_i, and class C_i, plus its observation history. Its pose in the map frame is never stored; it is computed when needed as

$$T_{W\leftarrow O_{\{i}\}} = T_{W\leftarrow K_{\{i}\}} \cdot T_{K_{\{i}\leftarrow O_i\}} \qquad (4)$$

A new observation is associated with an object of the same class within 0.15 m, nearest first and one-to-one per frame. Rotation is excluded from association because zero-shot estimators often return poses flipped by 180° for box-like objects. Position is the running mean of observations and orientation the mode of observations within 30°. The existence probability drives the states tentative ($p = 0.7$ at creation, hidden), active (≥ 2 observations and $p > 0.6$), lost ($p < 0.3$), remembered ($p < 0.05$ with ≥ 5 observations), and deleted. A miss lowers p by Bayes' rule only if the object projects into the view *and* lies within the recognizer's detection range; lost and remembered objects stay association candidates, so a returning object regains its identifier. Of two same-class objects, one with five times fewer observations is hidden as a duplicate. On the office run, the map ends with eight active objects (14–29 observations each) and two hidden tentative ones.

5.2 | Loop-closure-aware update

When SLAM corrects its keyframes (Fig. 1, panel 6), the next keyframe list carries the corrected poses $T'_{W\leftarrow K_{\{i}\}}$, and every object is updated by

$$T'_{W\leftarrow O_{\{i}\}} = T'_{W\leftarrow K_{\{i}\}} \cdot T_{K_{\{i}\leftarrow O_i\}} \qquad (5)$$

Only keyframe poses change; $T_{K_{\{i}\leftarrow O_i\}}$ is never modified, so all objects anchored to a keyframe move with it. An object whose keyframe was culled is re-anchored to the nearest live keyframe of the previous list, and the map can be rebuilt from the observation history whenever keyframe poses change (75 ms for 308 frames). For display, all map objects are projected at 15 Hz into a buffered image with the SLAM pose of that image's time, so drawing is independent of the recognition rate.

6 | EXPERIMENTAL SETUP

Runs. Recorded sessions are replayed in real time (1×) on the two computers, each replaying its own sensor stream. The SLAM computer is a laptop with an Apple M4 Pro processor; the recognition computer has an Intel Core i9-12900HK and an NVIDIA GeForce RTX 3080 Ti Laptop GPU [verify per run]. The recognizer comparisons of Sections 7.3–7.4 replay the recordings on a desktop with an NVIDIA GeForce RTX 4090, with ORB-SLAM3 (2000 features) on the same computer. SLAM always builds its map from scratch. Indoor runs with eight objects: the dark figure-eight run (284 s), the 91 s run (96 map corrections), the bright-hall figure-eight run (151 s), the large figure-eight run (315 s; one repeat with a 3.05 m loop correction is used in Fig. 1), and the office figure-eight run (128 s). Public data: YCB-Video [8] (900 BOP test images and the 12 test videos), LM-O, TUD-L, and IC-BIN [25], and YCB-M [9] (32 scenes).

TABLE 1 Data used in the experiments.

Run / dataset	Length	Used for
Dark figure-eight	284 s	loop closure, 18 runs
91 s run	91 s, 96 corr.	anchoring, re-association
Bright-hall figure-eight	151 s	re-association
Large figure-eight	315 s, 40 runs	loop update (3.05 m)
Office figure-eight	128 s	Fig. 1, timing, map
YCB-Video [8]	900 images, 12 videos	recognition, live maps
LM-O, TUD-L, IC-BIN [25]	200/600/150 im.	recognition
YCB-M [9]	32 scenes	display

Metrics. *Deviation* is the distance of an observation, registered in the final map, from the median position of its object. The recognizer is queried for every object of the dataset, one answer per object per image. An answer is correct if ADD-S is below 10% of the object diameter; *found* is the share of ground-truth objects answered correctly, *precision* the share of answers that are correct, and *wrong per image* the number of incorrect answers per image. BOP AR uses the official toolkit [25]. For live maps, an object entry is wrong if no ground-truth object of its class lies within the correctness bound (YCB-Video) or within 20 cm of the reference position (indoor runs). The *on-screen match rate* is the share of visible ground-truth objects whose displayed box is correct.

7 | RESULTS

7.1 | Capture-time registration and keyframe anchoring

We replay 61 recorded live ORB-SLAM3 runs (8964 results; capture-to-arrival latency median 1.45 s, maximum 6.6 s) with poses, keyframe updates, and results in their actual arrival order, and register the same results in four ways: (a) camera pose at arrival; (b) capture-time pose stored in the map frame; (c) capture-time pose anchored to the keyframe current at arrival; (d) proposed, anchored to the capture-time reference keyframe (Table 2, Fig. 4).

TABLE 2 Deviation of repeated observations in the final map (61 runs).

Registration	Median (cm)	90th pct. (cm)	> 10 cm (%)	90th pct., corrected (cm)	> 10 cm, corrected (%)
(a) Arrival-time pose	9.67	31.94	48.2	294.0	63.8
(b) Capture time, map frame	3.08	9.70	9.2	299.8	30.8
(c) Capture time, arrival-time keyframe	1.59	5.36	3.6	7.80	5.4
(d) Capture-time keyframe (proposed)	**1.51**	**5.21**	**3.5**	**5.81**	**1.5**

Corrected: the 130 results whose wait spanned a map correction larger than 2 cm.

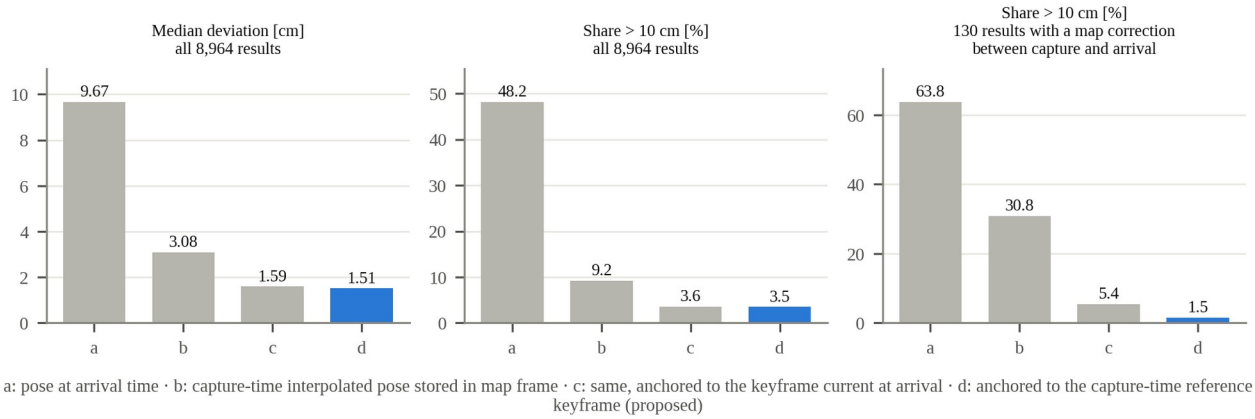


FIGURE 4 Registration strategies (a)–(d) on 61 replayed live runs.

Capture time removes most of the error (a → b). Without corrections in flight, (c) and (d) are similar, but among the 130 results whose wait spanned a map correction only (d) stays consistent (30.8% → 5.4% → 1.5% above 10 cm). The office run shows the same pattern with small corrections (maximum 11 cm): over 182 results, the share above 10 cm is 63.7%, 13.7%, 7.7%, and 6.6% for (a)–(d), and among the 7 results with a correction in flight it is 57% for (b) and (c) but 14% for (d). Across a loop closure in the dark figure-eight run, the position difference of the same object before and after the loop falls from 5.4 cm to 0.8 cm. The waiting queue raises the share of results placed on the map from 46% to 100%.

7.2 | Loop-closure update and persistent memory

In the large figure-eight run of Fig. 1 (panel 6), a loop closure moved keyframes by up to 3.05 m; storing the same 119 results in the map frame leaves 68 more than 10 cm from the final object positions, versus 6 with keyframe anchoring. In the 91 s run, 96 corrections of up to 35 cm occur; without anchoring, three objects leave copies 30–32 cm away (11 locations for 8 objects). Persistent memory re-associates 72 of 74 reappearing objects in two indoor runs, versus 34 when association is done in camera coordinates (Table 3); the two failures occur before a loop closure, when drift displaces the object beyond the association radius.

TABLE 3 Re-association in indoor runs (object map / without map).

	Bright hall, 151 s	91 s run
Reappearances (out of view)	35 (18)	39 (21)
Re-associated	35 / 22	37 / 12
Identifiers created (8 objects)	8 / 51	10 / 68
Same-class duplicates after run	0 / 17	0 / 19

Majority-vote orientation reduces displays off by more than 20° or 50 mm from 23.9% to 9.3%, duplicate suppression hides high-confidence misreadings (observation ratios 65:3, 72:3, 68:4), and range-aware miss counting raises the display ratio of a small mug and can from 37%/46% to 83%/82%.

7.3 | Recognition

Table 4 compares four recognizers on the 900 YCB-Video BOP test images on one RTX 4090. Our re-run of SAM-6D reaches BOP AR 82.8 (83.4 reported [3]). Replacing ViT-L by ViT-S and raising the selection score to 0.5 makes the recognizer faster, but it still answers for every selected proposal. Adding verification cuts wrong

answers per image from 0.232 to 0.049 and raises precision from 94.5% to 98.7%; of the 414 answers it removes, 165 are wrong. Cluster-first verification gives the same answers (identical pose for 99% of objects) at 240 ms per image, 45% of SAM-6D's time. Relative to SAM-6D, wrong answers fall to one sixth and found objects drop from 92.4% to 80.9%; BOP AR and ADD-S AUC count missed objects as zero, so they fall with the found rate. On the 3378 objects that both answer, the verified pose is more accurate (ADD-S AUC 94.6 → 96.7, median error 1.79 → 1.73 mm), because the answer is a consensus center rather than the geometric best.

TABLE 4 Single-image comparison on YCB-Video (900 BOP test images, 4123 objects, one RTX 4090).

Method	Abstains	Found (%)	Precision (%)	Wrong per image	BOP AR	Time (s)
GigaPose, coarse [5]	no	22.9	22.9	3.53	27.9	0.49
FoundationPose, 5 refinements [14]	no	93.1	95.3	0.21	86.2	4.01
SAM-6D [3] (FastSAM-x, ViT-L)	no	92.4	93.2	0.308	82.8	0.53
SAM-6D, ViT-S, score 0.5	no	86.9	94.5	0.232	78.1	0.33
+ verification	yes	80.8	98.7	0.049	74.1	0.34
+ verification, cluster-first (ours)	yes	80.9	**98.6**	**0.051**	74.1	**0.24**

GigaPose (RGB) and FoundationPose use CNOS-FastSAM detections (0.19 s, included). AR, average recall.

Only FoundationPose, which refines every pose five times by render-and-compare, finds more objects with fewer wrong answers than SAM-6D, at 16 times our time. Lowering the consensus share o from 0.3 to 0.15 recovers 140 correct answers (found 84.3%) for 11 more wrong ones (0.063 per image); we keep 0.3. Half of all abstentions come from insufficient consensus and the rest from the texture and silhouette checks. Table 5 repeats the comparison on three further BOP datasets on an RTX PRO 6000: precision stays at 97.9%–100% (SAM-6D 77.2%–97.5%), at a third to a quarter of the time. On that GPU the YCB-Video times are 495 ms and 185 ms.

TABLE 5 SAM-6D / ours on LM-O (200 images), TUD-L (600), and IC-BIN (150), RTX PRO 6000.

Dataset	Found (%)	Precision (%)	Wrong / image	Time (ms)
LM-O	71.9 / 45.3	77.2 / 97.9	1.53 / 0.07	894 / 267
TUD-L	84.5 / 61.3	85.8 / 99.2	0.14 / 0.005	267 / 94
IC-BIN	97.5 / 81.0	97.5 / 100	0.033 / 0	398 / 112

7.4 | Real-time maps and display

On the office run, SLAM tracks all 3844 input frames (11.6 ms each, 0 dropped) while recognition runs at 2.1 Hz with results arriving 1.57 s late (median), and no result remains unplaced. Drawing the object map at the display-time pose raises the on-screen match rate from 43.3% to 85.7% on YCB-Video and from 3.2% to 72.7% on YCB-M (Fig. 5).

Table 6 compares the final object maps built live with SAM-6D (score 0.5) and our recognizer, three runs each with the same SLAM and map. On the large figure-eight run, our map holds all eight objects and no wrong entry in every run, whereas SAM-6D misses two objects and leaves three wrong entries in two of three runs; recognition runs twice as fast (7.54 versus 3.63 Hz). On YCB-Video, both find nearly all 55 objects and the wrong entries differ little (43.7 versus 46.7, overlapping ranges): absent-object answers with plausible poses pass verification, and the faster recognizer processes 78% more frames. With verification off on the same front end, a single run leaves 173 wrong entries.

TABLE 6 Final object maps, live playback, three runs each (mean, range).

Data / method	Rate (Hz)	Objects found	Wrong entries	Map ADD-S AUC	Mean score (%)
Large figure-eight (8) · SAM-6D	3.63	6.7 (6–8)	2.3 (1–3)	–	55.7
Large figure-eight (8) · ours	**7.54**	**8.0 (8–8)**	**0.0 (0–0)**	**–**	**78.9**
YCB-V, 12 videos (55) · SAM-6D	1.76	53.7 (53–54)	46.7 (44–49)	89.2	52.8*
YCB-V, 12 videos (55) · ours	3.14	54.0 (54–54)	43.7 (42–47)	90.7	54.8*

Mean score: on-screen match rate averaged over the run; * at the end of the video. RTX 4090.

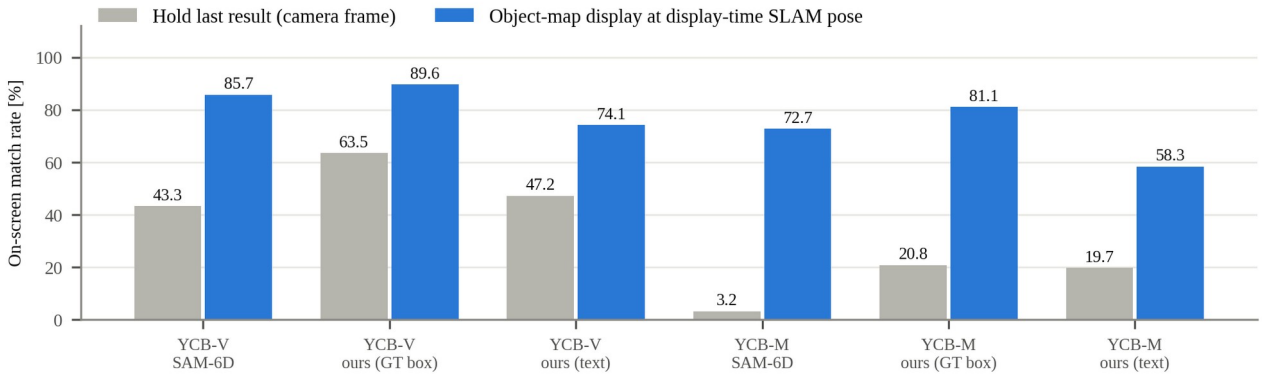


FIGURE 5 On-screen match rate: last result held in camera coordinates versus the object map drawn at the display-time SLAM pose.

7.5 | Limitations

Anchoring follows SLAM corrections, including wrong ones: in 40 runs of the large figure-eight with identical input, two loop closures damaged an accurate map. Abstention costs recall, more on textureless objects: found objects drop by 12 points on YCB-Video but by 23 –27 on LM-O and TUD-L, where most abstentions come from the texture check. On YCB-Video live playback, verification does not remove plausible answers for absent objects. Scenes with several objects of one class were not evaluated, and indoor runs lack ground-truth object poses.

8 | DISCUSSION

The two stages answer three questions that are often conflated. *Which time* registers the result: replacing arrival time with capture time accounts for most of the improvement (9.67 → 3.08 cm), because the platform moves while the result is computed. *Which frame* stores the object: storing it relative to a keyframe instead of the map frame is what lets it follow later corrections. *Which keyframe*: the keyframe current at arrival is adequate most of the time but fails when a correction happens in flight, because it pairs a pre-correction camera pose with a post-correction keyframe pose; the capture-time keyframe avoids this by construction (30.8% → 5.4% → 1.5% above 10 cm). Since zero-shot recognizers are slow and their latency varies, corrections in flight are not rare: 130 of 8964 results in our runs, concentrated around loop closures, where an object map is most likely to break.

The SLAM side only exposes information it already maintains, the reference keyframe of each pose and the poses of all keyframes, and schedules their transmission. The method therefore applies to other keyframe- or graph-based SLAM systems, as shown with ORB-SLAM3 and HDL-Graph-SLAM, and to other recognizers that output camera-frame poses. The recognizer was designed for this setting: in an object map a wrong answer leaves a persistent error, whereas a missed detection is recovered when the object is seen again, so precision is favored over per-frame recall. Verification adds no training and leaves SAM-6D unchanged; it decides whether to answer, and cluster-first verification makes that decision cheaper than the ViT-L features it replaces.

9 | CONCLUSION

This study maps late zero-shot 6D pose estimates consistently on an asynchronous SLAM map in two stages: capture-time registration, which pairs each result with the SLAM pose and reference keyframe of its capture time, and a keyframe-anchored object state map, in which a loop closure updates only keyframe poses while each object's relative pose $T_{K \leftarrow O}$ stays fixed. The separation of capture time, keyframe-relative storage, and keyframe choice shows that the last matters exactly when a correction happens in flight. A recognizer that verifies its pose hypotheses and may abstain keeps wrong answers out of the map at real-time rates. Future work includes detecting wrong loop closures, texture-adaptive verification, and ground-truth object poses for indoor runs.

REFERENCES

1. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, *ORB-SLAM3: an accurate open-source library for visual, visual–inertial, and multimap SLAM*, IEEE Trans. Robot. **37** (2021), no. 6, 1874–1890.
2. K. Koide, J. Miura, and E. Menegatti, *A portable three-dimensional LIDAR-based system for long-term and wide-area people behavior measurement*, Int. J. Adv. Robot. Syst. **16** (2019), no. 2.
3. J. Lin, L. Liu, D. Lu, and K. Jia, *SAM-6D: segment anything model meets zero-shot 6D object pose estimation*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024, pp. 27906–27916.
4. E. P. Örnek, Y. Labbé, B. Tekin, L. Ma, C. Keskin, C. Forster, and T. Hodaň, *FoundPose: unseen object pose estimation with foundation features*, (Proc. Eur. Conf. Comput. Vis., Milan, Italy), 2024.
5. V. N. Nguyen, T. Groueix, M. Salzmann, and V. Lepetit, *GigaPose: fast and robust novel object pose estimation via one correspondence*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
6. J. McCormac, R. Clark, M. Bloesch, A. J. Davison, and S. Leutenegger, *Fusion++: volumetric object-level SLAM*, (Proc. Int. Conf. 3D Vis., Verona, Italy), 2018.
7. T. Foote, *tf: the transform library*, (Proc. IEEE Int. Conf. Technol. Pract. Robot Appl., Woburn, MA, USA), 2013.
8. Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, *PoseCNN: a convolutional neural network for 6D object pose estimation in cluttered scenes*, (Proc. Robot. Sci. Syst., Pittsburgh, PA, USA), 2018.
9. T. Grenzdörffer, M. Günther, and J. Hertzberg, *YCB-M: a multi-camera RGB-D dataset for object recognition and 6DoF pose estimation*, (Proc. IEEE Int. Conf. Robot. Autom., Paris, France), 2020.

10. V. N. Nguyen, T. Groueix, G. Ponimatkin, V. Lepetit, and T. Hodaň, *CNOS: a strong baseline for CAD-based novel object segmentation*, (Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops, Paris, France), 2023.
11. A. Kirillov et al., *Segment anything*, (Proc. IEEE/CVF Int. Conf. Comput. Vis., Paris, France), 2023.
12. X. Zhao et al., *Fast segment anything*, arXiv preprint, 2023. https://arxiv.org/abs/2306.12156
13. M. Oquab et al., *DINOv2: learning robust visual features without supervision*, Trans. Mach. Learn. Res. (2024).
14. B. Wen, W. Yang, J. Kautz, and S. Birchfield, *FoundationPose: unified 6D pose estimation and tracking of novel objects*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
15. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, *YOLO-World: real-time open-vocabulary object detection*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
16. R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. J. Kelly, and A. J. Davison, *SLAM++: simultaneous localisation and mapping at the level of objects*, (Proc. IEEE Conf. Comput. Vis. Pattern Recognit., Portland, OR, USA), 2013.
17. S. Yang and S. Scherer, *CubeSLAM: monocular 3-D object SLAM*, IEEE Trans. Robot. **35** (2019), no. 4, 925–938.
18. L. Nicholson, M. Milford, and N. Sünderhauf, *QuadricSLAM: dual quadrics from object detections as landmarks in object-oriented SLAM*, IEEE Robot. Autom. Lett. **4** (2019), no. 1, 1–8.
19. Y. Labbé, J. Carpentier, M. Aubry, and J. Sivic, *CosyPose: consistent multi-view multi-object 6D pose estimation*, (Proc. Eur. Conf. Comput. Vis., Glasgow, UK), 2020.
20. J. Qian, V. Chatrath, J. Yang, J. Servos, A. P. Schoellig, and S. L. Waslander, *POCD: probabilistic object-level change detection and volumetric mapping in semi-static scenes*, (Proc. Robot. Sci. Syst., New York, NY, USA), 2022.
21. L. Schmid, M. Abate, Y. Chang, and L. Carlone, *Khronos: a unified approach for spatio-temporal metric-semantic SLAM in dynamic environments*, (Proc. Robot. Sci. Syst., Delft, Netherlands), 2024.
22. M. Rünz, M. Buffier, and L. Agapito, *MaskFusion: real-time recognition, tracking and reconstruction of multiple moving objects*, (Proc. IEEE Int. Symp. Mixed Augment. Real., Munich, Germany), 2018.
23. L. Liu, H. Li, and M. Gruteser, *Edge assisted real-time object detection for mobile augmented reality*, (Proc. Annu. Int. Conf. Mob. Comput. Netw., Los Cabos, Mexico), 2019.
24. C. Zhang et al., *Faster segment anything: towards lightweight SAM for mobile applications*, arXiv preprint, 2023. https://arxiv.org/abs/2306.14289
25. T. Hodaň et al., *BOP: benchmark for 6D object pose estimation*, (Proc. Eur. Conf. Comput. Vis., Munich, Germany), 2018.